The Effects of a Dynamic Traffic Light Control System on Traffic Conditions in Enniskillen and on Fermanagh Community Transport

Andrew Jones

October 10, 2026

1 Introduction

Enniskillen is Fermanagh's economic and education centre, providing goods and services to all througout the county. However, many residents of the county are not able utilize their own private transport, and therefore rely on public transport. Public transport in Fermanagh is underfunded and lacking, so many who are rurally or socially isolated rely on others or an organization called Fermanagh Community Transport (FCT) to be able to live their day to day lives or partake in leisure activities. FCT provides a door-to-door service to an estimated 1000 people in the local area [1].

This service is impacted by the large school and work rush hours causing congestion as most roads in the county radiate out from the main town, and most have to pass through it to get home from school or work as Lough Erne causes a bottleneck on the island town. Any infrastructure work being done in the town causes massive delays for the residents of the town [5]. There are potential solutions for this such as staggering work and school time endings, which has been done by the Enniskillen Royal Grammar School and Erne Integrated College to help combat this, or by using Vehicle Actuated dynamic traffic light systems to manage traffic. These dynamic traffic lights adjust their signal lengths based on real-time traffic conditions by using sensors to detect traffic density and or the number of pedestrians crossing the road, unlike fixed time traffic lights which only have set light phases.

In this project, we will be investigating the role of these dynamic traffic lights in managing traffic and how they can be potentially modified to reduce the likelihood for traffic jams by simulating fixed time traffic lights and dynamic traffic lights under the same conditons and comparing them to each other to see which is most effective during school and work rush hours in the town. This could potentially help FCT run more smoothly by reducing the amount of time their vehicles are stuck in traffic as well as reducing average waiting times for all in the town.

Firstly, section 2 will explain how our simulation works by using exponential random variables to generate arrival times, and a flowchart to explain how our algorithm simulates

and records data to be used in figures to visualize their effectiveness of the two types of lights. Next, section 3 will compare the results of these simulations and use our findings to discuss potential modifications to the traffic light system of Enniskillen and the effect it will have on FCT and commuters in the County. Finally, we will conclude that, while this proposal is reasonable and likely to have a positive affect, more should be added to the model to gain more of an insight into the affects of this new traffic light system, and potential extentions to the algorithm using examples from different parts of the world.

2 Model

A traffic light is a real time application of a circular queue that follows the first in first out principle as the red light and green light rotate between each other in regular intervals. A traffic light is unique in the sense that it has a linear queue inside of a circular queue, and this linear queue can only be served whenever the circular queue has served the red light and is now serving the green light. A circular queue is an advanced version of a regular queue where the last element is connected back to the first element, forming a circular structure [4]. Figure 1 shows an algorithm that can be used to simulate a single traffic light in one direction, with an altering traffic condition depending on the clock time in relation to the time of day and a dynamic light system that responds to the traffic conditions at the end of each light phase. These functions can be turned off to make the dynamic traffic light a fixed time traffic light.

The simulation takes in the base light duration (for both red and green, denoted as μ), the time it takes for a vehicle to depart from the light (denoted t_car), the traffic conditions of the day in the form of a rate function over time which then gives out the average car arrival rate for that hour (denoted $g(t)$), the light phase function over the number of vehicles in the system to give the light phase multiplier for the dynamic traffic light system (denoted $f(N)$, and finally the clock time the simulation will run to (t_max). The clock time/clock value of the simulation is denoted as t. Furthermore, to ensure that t is between 0 and 600 for g (and thus telling us the time of day), we can do the following transformation

$$t' = t - 600 * \text{floor}(\frac{t}{600}) \tag{1}$$

This ensures that we subtract 600 minutes from the clock time if the simulation clock runs over 600 into day 2 for the function g.

In the simulation used to produce Figures 2, 3, and 4, the variables used were as follows:

- $\mu = 1$. This represents a base light duration of one minute for both green and red phases.
- t_car $= 1/30$. This represents a car departure time of 2 seconds from the traffic light system.

- $$g(t') = \begin{cases} 10, & 180 \leq t' < 240, \\ 15, & 240 \leq t' \leq 360, \\ 5, & \text{otherwise.} \end{cases} \tag{2}$$

  . This represents an average vehicle arrival rate of 10 per minute for 3pm-4pm, 15 per minute for 4pm-6pm, and 5 per minute for the rest of the day (t' = 0 represents 12pm, t' = 60 represents 1pm and so on).

- $$f(N) = \begin{cases} 1, & N < 5, \\ 1.5, & 5 \leq N < 10, \\ 2, & 10 \leq N < 15, \\ 2.5, & 15 \leq N < 20, \\ 3, & 20 \leq N < 25, \\ 3.5, & 25 \leq N < 30, \\ 4, & \text{otherwise.} \end{cases} \tag{3}$$

  . This gives an increasing light phase multiplier to increase the green light duration and reduce the red light duration at abnormally high traffic levels.

- t_max = 600. This represents a simulation over a 10 hour day from 12pm-10pm. Once this number is reached the simulation ends.

In the beginning of this simulation, the initial variable values are set. The simulation clock starts at 0, the light starts at red and the next green light time is generated by adding the base light duration onto the simulation clock.

An exponential random variable with parameter $\frac{1}{g(t')}$ is then generated and added onto the simulation clock to find the next time of arrival. At this arrival time, the number of cars in the system (i.e stuck behind the traffic light) increases by one. Another exponential random variable is then generated and added onto the simulation clock to find the next arrival time.

This process is then repeated until the simulation clock is at or exceeds the green light time, where the number of cars in the system N is recorded into a list and then the light turns green. The next red light time is then generated using the equation

$$\text{t_red} = t + \mu f(N) \tag{4}$$

and the next departure is generated by adding t_car to the green light time. Once the simulation clock reaches the next departure time, t_car is added to the simulation clock again and the number of cars in the system decreases by 1.

While the light is green, arrivals still occur if the next arrival time is less than the next departure time, in which a new arrival time is generated and N increases by 1. Once the clock exceeds t_red, the next green light time is generated using the equation

$$\text{t_green} = t + \frac{\mu}{f(N)} \tag{5}$$

and the light turns red. This whole process is then repeated until the clock exceeds the max time value, where the simulation then ends.

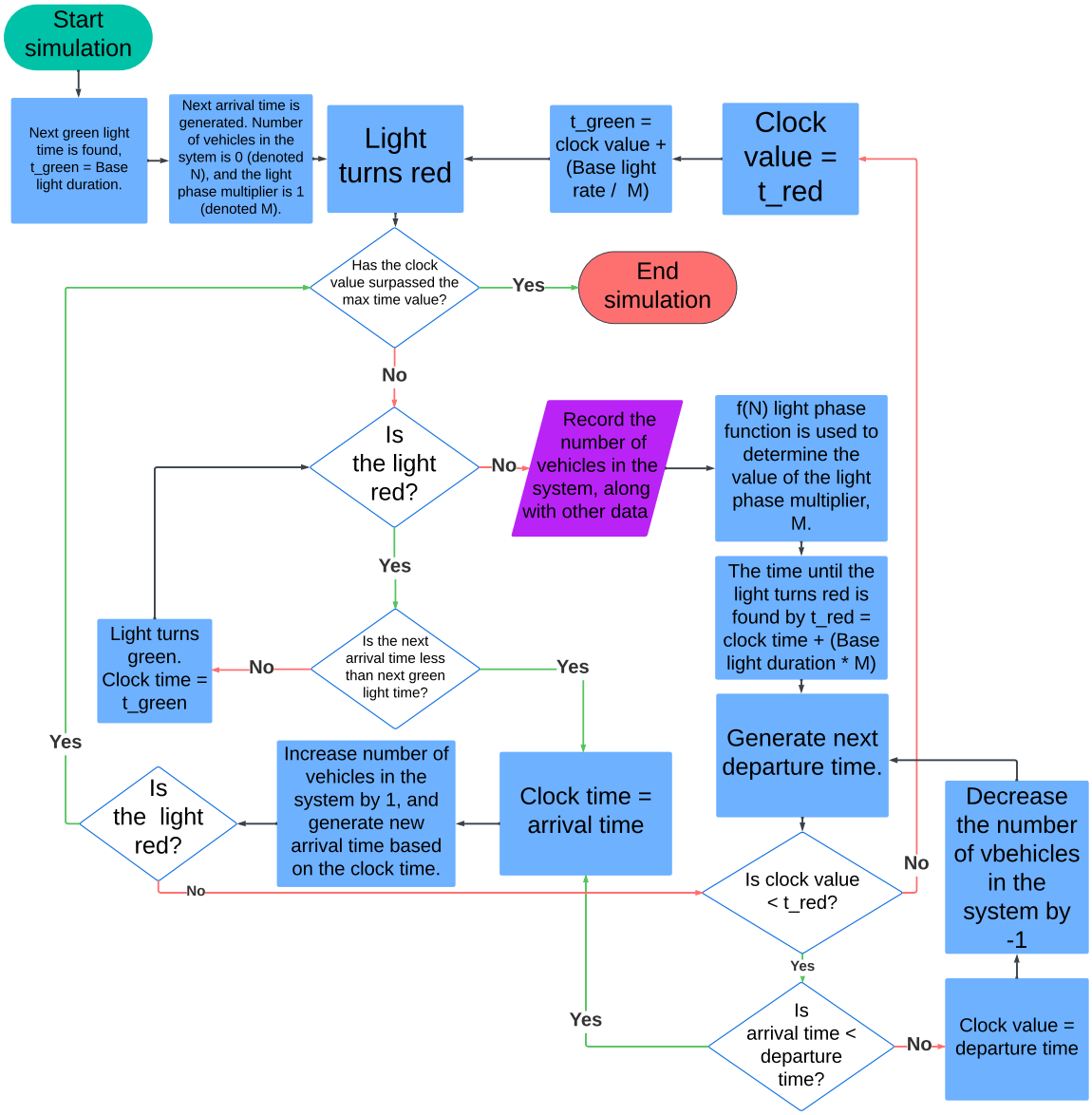


Figure 1: A flowchart of how the simulation of the dynamic traffic light works. The blue squares represent processes, the white triangles represent decisions, and the purple parallelogram represents the recording of data when the traffic light turns green.

This process is more easily visualised using Figure 6, which also shows how the clock is updated throughout the simulation. This flowchart is based on the code used to create the simulation and generate the following figures to be used in the model interpretation and proposal section, shown in the codes section.

For a fixed traffic light simulation, $f(N)$ is made to be the constant function returning 1 for all N.

Figure 6 shows how I updated this algorithm so that the traffic light continuously monitors the number of vehicles in the system to updat ehte light phase function, and I attempted to add a system to aid emergency vehicles by reducing their wait times.

3 Interpretation and Proposal

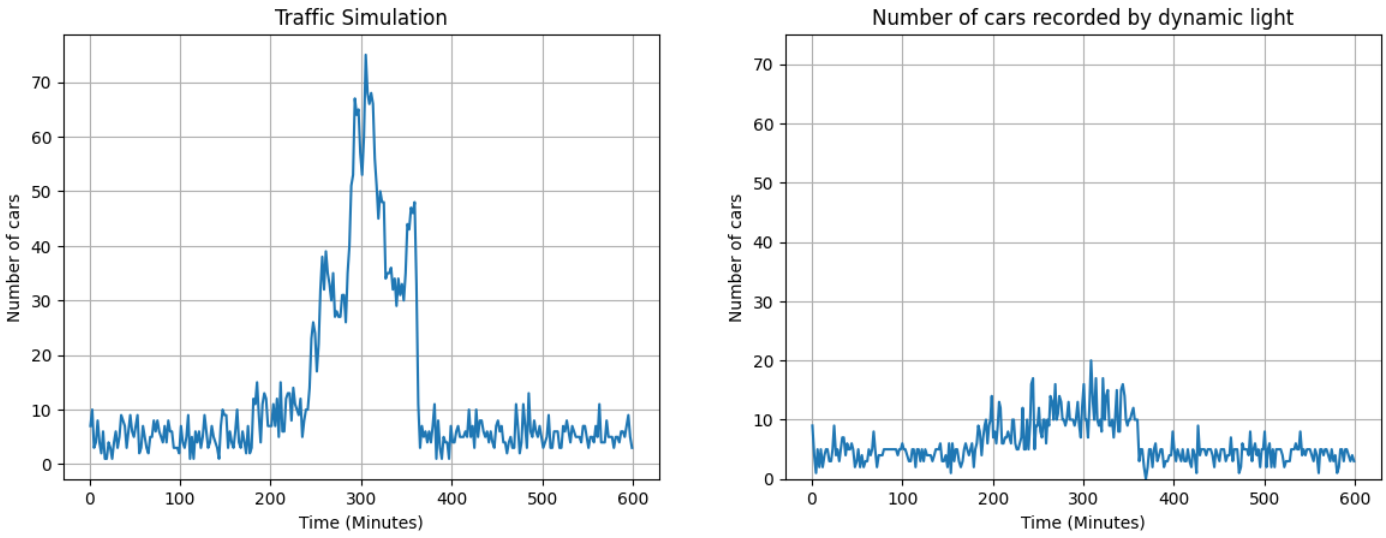


Figure 2: The number of cars recorded after simulating typical work day traffic from 12pm to 10pm when a fixed time traffic light turns green (left) and a dynamic traffic light turns green (right). The average number of cars arriving at the traffic light per minute was 5 for all times except between 3pm-5pm to simulate school and work rush hour traffic. The average number was increased to 10 cars per minute between 3pm-4pm, and 15 cars per minute between 4pm-5pm. The base light duration was 1 minute for both the red and green phases, and the cars left the light at a rate of one every 2 seconds while the light was green. 0 minutes on the x-axis represents 12pm, 60 minutes represent 1pm, and so on.

Figure 2 shows the comparison in the number of cars throughout the day between the dynamic traffic light and the fixed traffic light simulated with the same conditions as in 2. The number of cars peaks at around 70 for the fixed light and around 25 for the dynamic light. This demonstrates that the dynamic light is preferable for handling high amounts of traffic compared to the fixed light, which Enniskillen town mostly uses, such as the crossroad junction connecting Wellington Rd, Dublin Rd, Forthill st and Belmore st that is highly susceptible to congestion and thus causing congestion to the rest of the town.

Figure 2 was generated by recording the number of cars and the clock time every time the light turns green, and then plotting the number of cars in the system at each green light against its corresponding clock time.

If this junction were to implement a system/algorithm like this or one or the various adaptive and dynamic smart traffic light systems studied [2], such as the dynamic algorithm studied by Kumar et al [3], which uses queue length, entry rate, and exit rate as inputs to estimate the green traffic signal time allocated to each road in a simple 4-road intersection, then traffic in the whole town could be reduced and thus FCT wait times would be reduced.

The down side to the algorithm in 2 is that it only applies to a single traffic light in one direction, which could cause problems or be dangerous if applied to an intersection. It should therefore be extended to be able to handle intersections and multiple lights to see its effect.

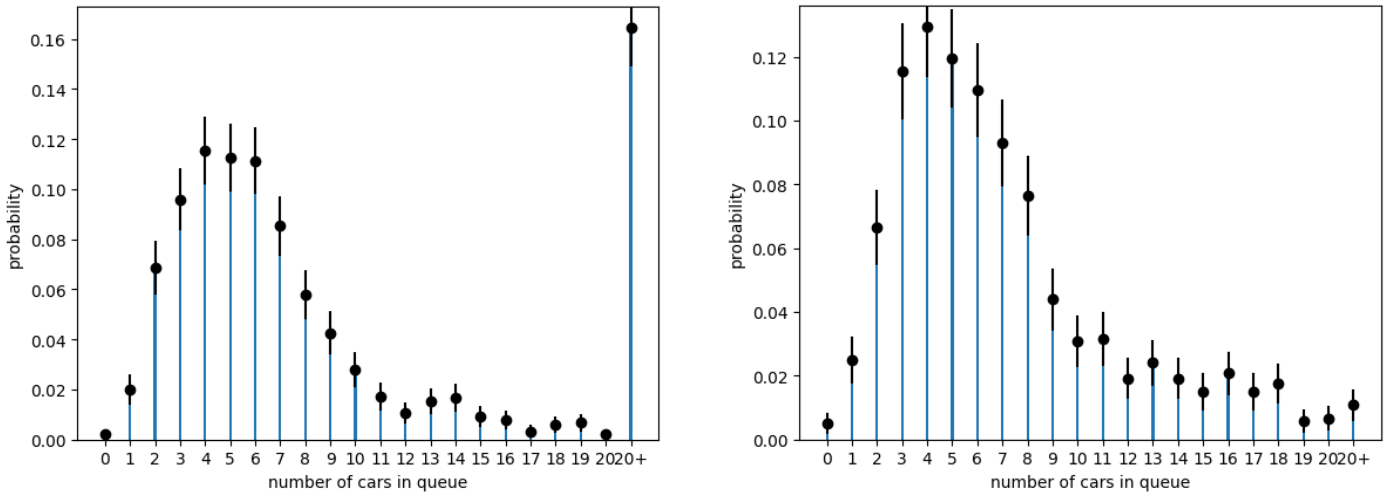


Figure 3: An Estimation of the distribution of the number of cars at a fixed time traffic light when it turns green (left) and a dynamic traffic light when it turns green (right) under the same conditions as Figure 2. A simulation of 5 days (A maximum of 3000 minutes in the simulation clock) was used to estimate the distribution, where black error bars are shown indicating a 95% confidence limit for the distribution.

Figure 3 shows the comparison in the distribution in the number of cars in the queue between the fixed and dynamic lights for the same previous simulation ran over 5 in simulation days (3000 minutes). The probabilities for the higher number of cars in the fixed light sim is higher than in the dynamic light sim, especially for 20+ cars. Both figures take on the same shape, apart from the huge increase in 20+ cars for the fixed light sim, suggesting that the fixed light performs poorly at higher traffic rates. This consolidates what was discussed in previous paragraphs.

Figure 3 was generated by dividing the number of cars into bins, and then adding 1 to a bin if the light turns green with the number of cars in the system corresponding to that bin. Once the simulation is finished, the bins are divided by the number of light changes to give the probability distribution that if you randomly select one instance of a light turning green at any time in the day, what will the number of cars behind the light be?

Figure 4 shows the comparison in the distribution of cars throughout time, which is a more simplified histogram version of Figure 2 and mirrors the results prior. The confidence limits suggest that we are 95% sure that these results accurately represent the traffic flow under these conditions for both lights.

Figure 4 was generated by dividing the time of day into bins (0 to 9, corresponding to 12pm-10pm), Storing the total number of cars recorded in the system during each hour and adding to the corresponding bin, recording the total number of light phases, and then normalizing by dividing the bins by the total number of phases to produce the probability distribution.

Figure 5 shows the comparison between the simulations for the fixed light and dynamic light using data collected at the location of a real traffic light in Enniskillen throughout

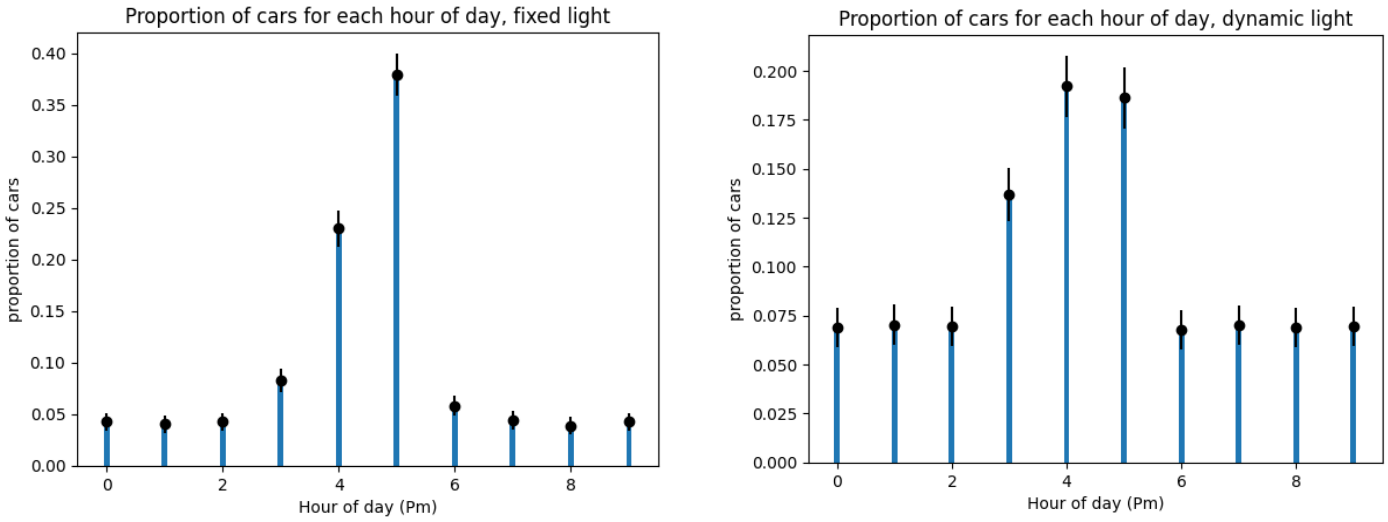


Figure 4: An Estimation of the distribution of the proportion of cars in traffic for times throughout the afternoon and evening hours at a fixed time traffic light when it turns green (left) and dynamic traffic light when it turns green (right) under the same conditions as Figures 2 and 3. A simulation of 5 working days (A maximum of 3000 minutes in the simulation clock) was used to estimate the distribution, where black error bars are shown indicating a 95% confidence limit for the distribution.

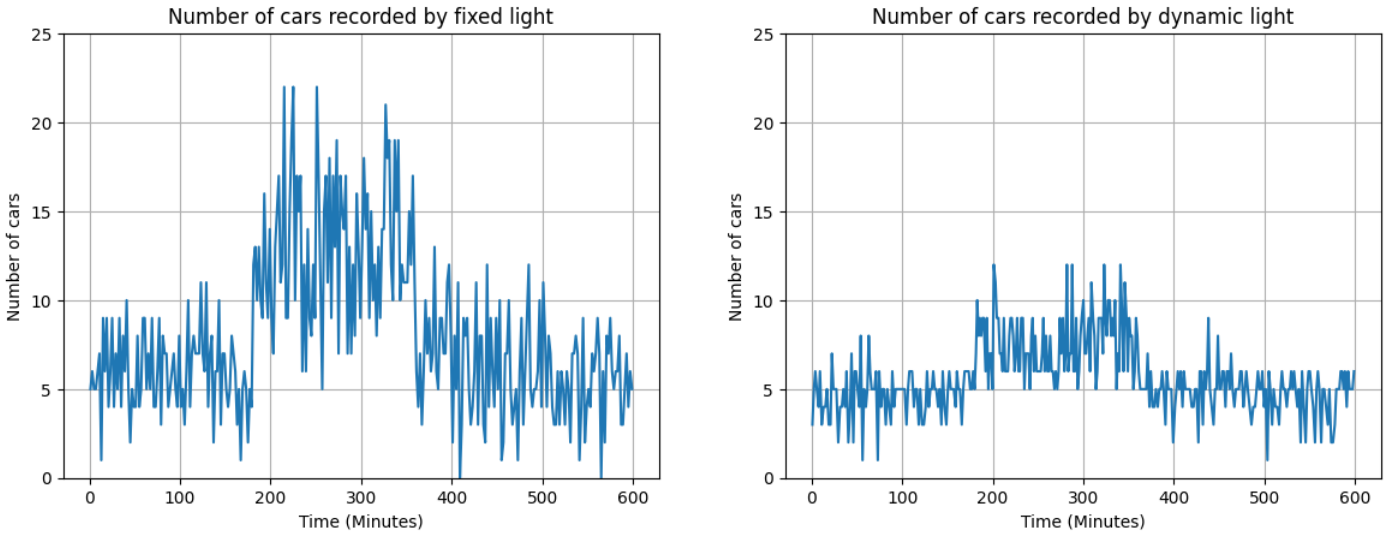


Figure 5: The number of cars recorded after simulating typical work day traffic from 12pm to 10pm when a fixed time traffic light turns green (left) and a dynamic traffic light turns green (right). The average number of cars arriving at the traffic light per minute was 6 for all times except between 3pm-5pm to simulate school and work rush hour traffic. The average number was increased to 12 cars per minute between 3pm-5pm to simulate real life data collected from the Enniskillen through-pass traffic light in 2015. The base light duration was 1 minute for both the red and green phases, and the cars left the light at a rate of one every 2 seconds while the light was green. 0 minutes on the x-axis represents 12pm, 60 minutes represent 1pm, and so on.

the year of 2015. This data set includes the direction of traffic (channels), the average 5 day traffic flow from Monday to Friday, and the peak AM and PM hour flow for each channel including peak times.

This traffic count data from 2015 comes along with summaries and a glossary. The traffic counter site of interest is site No. 646, located at the Enniskillen through pass next to

Cafe Maud's. The data shows that the average PM Peak for channel 2 (heading west out the town past the castle) was 710 vehicles per hour at 17:00. Dividing this number by 60 we get a peak vehicle arrival rate of around 12. It also shows that the average weekday count for channel 2 was 9100 vehicles, which corresponds to an average arrival rate of 6 cars per minute. Using this information, the simulation was redone by changing the rate functions to

$$f(N) = \begin{cases} 1, & N < 3, \\ 1.5, & 3 \leq N < 6, \\ 2, & 6 \leq N < 9, \\ 2.5, & 9 \leq N < 12, \\ 3, & 12 \leq N < 15, \\ 3.5, & 15 \leq N < 18, \\ 4, & \text{otherwise.} \end{cases} \tag{6}$$

and,

$$g(t') = \begin{cases} 12, & 180 \leq t' < 300, \\ 6, & \text{otherwise.} \end{cases} \tag{7}$$

. The rest of conditions were kept the same as in 2.

Figure 5 shows a more realistic day to day comparison between the fixed and dynamic light if the dynamic light was to be used in Enniskillen. It shows that the number of cars recorded at the end of each red light was halved, which would significantly reduce FCT bus and general vehicle waiting times. This would correspond to waiting times roughly being halved, which is similar to the results to Kumar et al's study on Adaptive traffic light control using deep reinforcement learning technique.

The work in Kumar et al's study is based on deep learning to solve the problem of traffic congestion in the urban environment. The model classifies the vehicle's type on the road and assigns different vehicle weights. The proposed work has trained using experience replay and target network based on a deep double-Q learning mechanism, where the proposed model in this study efficiently reduced the average waiting time up to 91.7 % at four road intersections [3].

4 Conclusions

In this report, we have shown how a dynamic traffic light in one direction could be modeled by creating an algorithm that takes the traffic movement speed, base light duration, and rate functions to return a light duration multiplier used to find the light duration for that particular phase shown in Figure 1.

We used simulations to demonstrate that a dynamic traffic light system is much better at handling traffic in one direction than a fixed light shown in Figures 2-4. Of course this is to be expected, as reducing the red light time and increasing the green light time will have no repercussions on any other traffic light or system in these simulations. This is why this algorithm should be extended to include other scenarios, such as a simple 4-way junction like the one connecting Dublin Rd, Wellington Rd, Belmore St and Forthill St.

One way is by giving certain traffic lights higher priority depending on factors like traffic flow rate, pedestrians, number of vehicles in queue, priority vehicles like ambulances as was implemented in systems by Zerroug et al and Kumar et al [3][2].

Real life data can then be used from traffic counters to be inputted into the simulation to get a more accurate representation of the algorithms performance in Enniskillen, which would be of great use to transport organizations that operate in Fermanagh like Translink and FCT.

One way that I would modify the algorithm is by monitoring the number of cars waiting and adjusting the length of the lights continuously instead of just at the end of light phases. For example, if a queue of cars behind a tractor appears or an ambulance in the middle of a red light, the light would become much shorter. Another way is by somehow making the algorithm create its own rate function to use as time progresses.

To conclude, the community partner FCT should make recommendations to Femanagh and Omagh district council and the Department for Infrastructure to look into the use of dynamic light systems for particular or all junctions in Enniskillen to reduce traffic congestion and thus reducing wait times for FCT users and all other commuters.

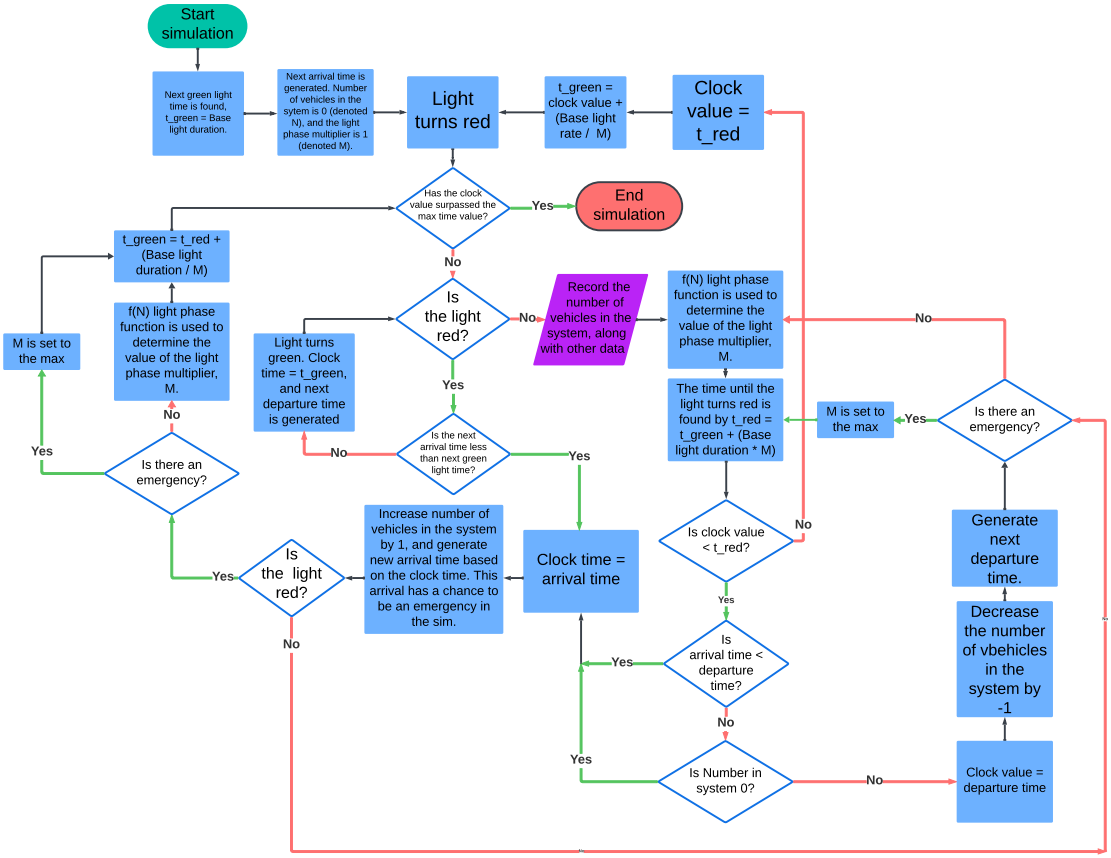


Figure 6: A flowchart of how the simulation of the updated dynamic traffic light works. The blue squares represent processes, the white triangles represent decisions, and the purple parallelogram represents the recording of data when the traffic light turns green.

References

[1] *Fermanagh community transport website* www.fermanaghcommunitytransport.com (accessed 07/01/25).

[2] *Adaptive and dynamic smart traffic light system for efficient management of regular and emergency vehicles at city intersection - Rafik Zerroug et al.* https://ietresearch.onlinelibrary.wiley.com/doi/full/10.1049/smc2.12090 (accessed 08/01/25)

[3] *Intelligent traffic controller - Sachin Kumar et al.* https://link.springer.com/article/10.1007/s41870-019-00405-8 (accessed 09/01/25).

[4] *Understanding Circular Queues - Medium - Manoj Kumar Bochu* https://medium.com/@manojkumarbochu/understanding-circular-queues-a41c6ec47740 (accessed 19/01/25).

[5] *Enniskillen traffic build up - Fermanagh Herald* https://fermanaghherald.com/2024/06/enniskillen-motorists-anger-at-town-traffic-build-up/ (accessed 19/01/25).

A Codes

```
#mu = base duration of green light and red light(service time)

#t_red = base duration of red light phase

#t_car = time it takes for vehicles to pass through light

#maxlen = maximum number of cars registered in the system for the
    simulation

#maxtime = maximum time on simulation clock that the simulation will
    run for

class Simulation :
    def __init__(self, mu, t_car, maxlen, maxtime) :   #Function for
    setting initial values for all the variables
        self.maxtime = maxtime
        self.light_state = 'red'
        self.mu = mu

        self.t_car = t_car
        self.clock = 0.0
        self.t_arrival = self.generate_interarrival()  # Time until
    next car arrives (Constantly going for both light phases)
        self.t_green = mu  # t_green = time until next green light
```

```
21          self.t_individual_depart = mu + t_car  # Departure time for the
    next individual vehicle during green light
22          self.t_red = 0  # t_red = time until next red light
23          self.emergency = False #Represents no emergency vehicles in the
    system
24          self.queue_num = 0 #Represents the emergency vehicles queue
    number
25
26
27
28          #Initial for variables that take in information for the
    simulation (Some of these aren't needed)
29          self.traffic_history = []
30          self.time_history = []
31          self.time_with_length = np.zeros(maxlen+2)   #Initial list for
    finding proportion of traffic
32          self.number_list = []
33          self.num_in_system = 0
34          self.number_for_times = np.zeros(10)
35          self.num_arrivals = 0
36          self.num_departures = 0
37          self.num_changes = 0 #Number of light changes
38          self.total_wait = 0.0  #Total wait time in the simulation
39
40      def advance_time(self):
41        while self.clock <= self.maxtime:
42          if self.clock > self.maxtime:
43            break
44          t_event = min(self.t_arrival, self.t_green)
45          self.clock = t_event
46
47
48          if self.light_state == 'red':
49            self.handle_red_time()   #Red light phase
50
51          if self.light_state == 'green':
52            self.handle_green_time()
53              #Green light phase
54
55
56
57      def handle_red_time(self) :
58
59          if self.t_arrival <= self.t_green :
60              self.handle_arrival_event()
61          else :
62              self.light_state = 'green'
63
64          self.t_green = self.t_red + self.mu*(1/self.light_rate())
65
66      def handle_green_time(self):
67          self.t_red = self.clock + self.mu*self.light_rate() #Time until
    next red light phase
68
69             #Note:
70             #Self.clock = current simulation time
71             # Self.light_rate() = multiplier for the current light cycle
    depending on traffic conditions
```

```
72            #Self.mu = base green light duration
73            # Self.t_red = time of next red light
74          self.check = 1
75        #Info to help with making figures
76          self.traffic_history.append(self.num_in_system)
77
78          if self.num_in_system <= maxlen:
79            self.time_with_length[self.num_in_system] += 1
80
81
82          if self.num_in_system > maxlen:
83            self.time_with_length[maxlen+1] += 1
84
85          self.time_history.append(self.clock)
86
87          self.number_list = np.append(self.number_list, self.
    num_in_system)
88
89          self.num_changes += 1
90          for ii in np.arange(0,10,1):
91                      if self.traffic_rate(self.clock)[0] == ii:
92                         self.number_for_times[ii] += self.num_in_system
93
94        #Actual arrival/departure events for a green light
95
96
97
98
99          while self.clock < self.t_red:
100
101           t_event = min(self.t_arrival, self.t_individual_depart)
102           self.total_wait += (self.num_in_system - 1) * (t_event - self
    .clock)
103           self.clock = t_event
104
105           if self.clock >= self.t_red:
106             break
107
108           if t_event == self.t_arrival or self.num_in_system == 0:
109             self.handle_arrival_event()
110
111
112           else:
113             self.handle_depart_event()
114
115           self.t_red = self.t_green + self.mu*self.light_rate()
116
117         self.clock = self.t_red
118         self.t_green = self.clock + self.mu*(1/self.light_rate())
119         self.t_individual_depart = self.t_green + self.t_car
120         self.light_state = 'red'
121
122     def handle_arrival_event(self) :
123
124         self.num_arrivals += 1
125         self.num_in_system += 1
126
127         #Emergency situation
```

```
        self.Emergency_var = np.random.choice([0, 1], p=[1-0.005, 0.005])

        if self.Emergency_var == 1:
            self.emergency = True
            self.queue_num = self.num_in_system
            self.ambulance_arrival_time = self.clock

        print(f"Clock: {self.clock}, traffic: {self.num_in_system} t_green: {self.t_green} t_red: {self.t_red}, t_arrival: {self.t_arrival}, t_individual_depart: {self.t_individual_depart}")
        print(f"light colour: {self.light_state} arrival, light phase multiplier {self.light_rate()}, emergency = {self.emergency}")

        self.t_arrival = self.clock + self.generate_interarrival()


    def handle_depart_event(self):



        if self.num_in_system > 0:

          print(f"Clock: {self.clock}, traffic: {self.num_in_system} t_green: {self.t_green} t_red: {self.t_red}, t_arrival: {self.t_arrival}, t_individual_depart: {self.t_individual_depart}")
          print(f"light colour: {self.light_state} departure, light phase multiplier {self.light_rate()}, emergency = {self.emergency}")

          self.num_in_system -= 1
          self.num_departures += 1

          if self.emergency == True:
            self.queue_num -= 1

            if self.queue_num == 0:
              self.emergency = False

          self.t_individual_depart = self.clock + self.t_car




    def traffic_rate(self,t) :
        # Calculate t' to represent time of day (0-600 minutes)
       t_prime = t - 600 * np.floor(t / 600)

    # Define rate based on time intervals

       if 0 <= t_prime < 60:
         return 0,5  # Rate for 0:00-1:00
       elif 60 <= t_prime < 120:
         return 1,5  # Rate for 1:00-2:00
       elif 120 <= t_prime < 180:
         return 2,5  # Rate for 2:00-3:00
       elif 180 <= t_prime < 240:
         return 3,10  # Rate for 3:00-4:00
       elif 240 <= t_prime < 300:
```

```
        return 4,15 # Rate for 4:00-5:00
      elif 300 <= t_prime < 360:
        return 5,15  # Rate for 5:00-6:00
      elif 360 <= t_prime < 420:
        return 6,5  # Rate for 6:00-7:00
      elif 420 <= t_prime < 480:
        return 7,5  # Rate for 7:00-8:00
      elif 480 <= t_prime < 540:
        return 8,5  # Rate for 8:00-9:00
      elif 540 <= t_prime < 600:
        return 9,5  # Rate for 9:00-10:00
      else:
        return 5 # Default rate if not in any specified interval

  def light_rate(self):


      if self.num_in_system < 5:
        return 1
      elif 5 <= self.num_in_system < 10:      # Return 1 for all for
no light phase adjustment
        return 1.5
      elif 10 <= self.num_in_system < 15:
        return 2
      elif 15 <= self.num_in_system < 20:
        return 2.5
      elif 20 <= self.num_in_system < 25:
        return 3
      elif 25 <= self.num_in_system < 30:
        return 3.5
      else:
        return 4


  def generate_interarrival(self) :
      return np.random.exponential(1/self.traffic_rate(self.clock)
[1])

  #Functions to retrieve information from the simulation

  def get_fraction_of_time_with_length(self):
      return self.time_with_length / self.num_changes

  def get_time_history(self):
      return self.time_history

  def get_traffic_history(self):
      return self.traffic_history

  def queue_size_list(self):
      return self.number_list

  def number_departures(self):
      return self.num_departures

  def sclock(self):
      return self.clock

```

```
235     def num_change(self):
236         return self.num_changes
237 
238     def num_times(self):
239         return self.number_for_times
240 
241     def get_check(self):
242       return self.check
243 
244 
245 
246 # Run a simulation until 600 minutes is reached (10 hours)
247 
248 #Here base light and red phase = 1 min
249 #nevents = 10000 to run the simulation until maxtime
250 # Maxlen = 20 so the maximum number of cars recorded for the proportion
      figures is 20
251 # t_car = 2/20 so it takes each car 2 seconds to exit the light system
     when green
252 # maxtime = 600 so the simulation emulates a working day from 12pm to 8
     pm
253 
254 expr, nevents, maxlen, t_car, maxtime = 1, 10000, 20, 2/60, 3000   #
     Change maxtime to 3000 to simulate over 5 days
255 s = Simulation(expr, t_car, maxlen, maxtime)                       #for
     the proportion figures
256 
257 
258 
259 
260 
261 for i in range(nevents):
262   s.advance_time()
263 
264 traffic_history = s.get_traffic_history()
265 
266 time_history = s.get_time_history()
267 
268 
269 #Figure for number of cars at the end of each red light phase
     throughout the day
270 plt.figure()
271 plt.plot(time_history, traffic_history)
272 plt.ylim(0, 75)
273 plt.xlabel("Time (Minutes)")
274 plt.ylabel("Number of cars")
275 plt.title("Number of cars recorded by dynamic light")
276 plt.grid(True)
277 
278 print(np.arange(0,10,1))
279 #Figure for proprotion of vehicles for each hour of the day
280 indices, times, nchanges = np.arange( 0, 10, 1 ), s.num_times(), s.
     num_change()
281 times = times / sum(times)
282 
283 
284 #Calculating error bar
285 
```

```
error = np.sqrt((times * (1 - times)) / ( nchanges - 1)) * scipy.stats.
    norm.ppf(0.95)
plt.figure()
plt.errorbar( indices, times, yerr=error, fmt='ko' )
plt.bar( indices, times, width=0.1 )
plt.xlabel("Hour of day (Pm)")
plt.ylabel("proportion of cars")
plt.title('Proportion of cars for each hour of day, dynamic light')


#Figure for proportion of number of vehicles found at the end of each
    red light
indices, times, queue_list, ndepartures, = np.linspace( 0, maxlen,
    maxlen + 1), s.get_fraction_of_time_with_length(), s.queue_size_list
    (), s.number_departures()

indices_float = np.append(indices, 21)

indices_string = [str(int(i)) for i in indices_float]  # Convert
    numeric indices to strings
indices_string[-1] = "20+"

nchanges = s.num_change()
print(np.linspace(0, 12, 13))

sum_times = 0
for ii in range(maxlen + 1):
  sum_times = sum_times + times[ii]


final_time = s.sclock()
error = np.sqrt((times * (1 - times)) / ( nchanges - 1)) * scipy.stats.
    norm.ppf(0.95)

plt.figure()
plt.errorbar( indices_float, times, yerr=error, fmt='ko' )
print("Number of departures", ndepartures)
print("Final time", final_time)
plt.bar( indices_string, times, width=0.1 )
plt.xlabel("number of cars in queue")
plt.ylabel("probability")
```